Електротехнички факултет
Универзитета у Београду



Лука Калуђеровић 2021/0041

Интелигентни *AMOS* систем за авио-компаније заснован на дистрибуираној сервисно-оријентисаној архитектури

ментор: доц. др Владимир Јоцовић

Београд, 2026.

Садржај

1. Увод 1
2. Преглед постојећих решења 3
2.1. AMOS (Swiss Aviation Software) 3
2.2. Ramco Aviation Suite 4
2.3. Trax Maintenance 4
2.4. OASES (Commsoft) 5
2.5. Компаративна анализа постојећих AMOS решења 6
2.6. Поређење са развијеним системом 7
3. Коришћене технологије 8
3.1. Flutter (програмски језик Dart) 8
3.2. ASP.NET (програмски језик C#) 8
3.3. ML.NET (програмски језик C#) 8
3.4. Microsoft SQL Server 9
3.5. JWT аутентификација и ауторизација 9
4. Архитектура система 10
4.1. Презентациони слој (frontend) 10
4.2. Апликативни слој (backend) 11
4.3. Слој података (database) 11
4.4. Слој интелигентних компоненти 12
4.5. Ток комуникације у систему 12
4.6. Безбедносни и скалабилни аспекти архитектуре 13
5. Имплементациони детаљи 14
5.1. Аутентификација и ауторизација корисника 14
5.2. Permission-based модел контроле приступа 14
5.3. Организација података у бази 15
5.4. Чување лозинки и безбедност података 16
5.5. Интеграција provider сервиса машинског учења 17
5.6. Тренирање модела машинског учења 17
6. Функционалности система 18
6.1. Аутентификација и кориснички профил 18
6.2. Управљање авионима 19
6.3. Управљање рутама 20
6.4. Управљање одржавањима 21
6.5. Административне функционалности 22
6.6. Контрола приступа и видљивост функционалности 23
7. Закључак 24

1. Увод

Савремена комерцијална авио-индустрија представља један од најсложенијих оперативних система данашњице. Авио-компаније свакодневно управљају великим бројем летелица, посада, рута, одржавања и регулаторних захтева, при чему се од њих очекује висок ниво безбедности, тачности и економске ефикасности. У таквом окружењу, информациони системи имају кључну улогу у обједињавању података и подршци у доношењу одлука.

Један од најзначајнијих информационих система у авио-компанијама јесте *AMOS* (енгл. *Aircraft Maintenance and Operational System*), који обједињује податке о флоти, рутама, одржавању, ресурсима и корисницима. Традиционални *AMOS* системи углавном су фокусирани на евиденцију и административну подршку, док се савремени трендови све више усмеравају ка интелигентним системима који користе методе машинског учења ради предикције и оптимизације пословних процеса.

Додатни изазов савременог пословања представља промена начина рада изазвана глобалном пандемијом *COVID-19*, након које је значајно порастао обим рада на даљину и ван класичног канцеларијског окружења. У том контексту, информациони системи више нису везани искључиво за десктоп рачунаре, већ се очекује њихова доступност на преносивим уређајима. Мобилни и таблет уређаји омогућавају инжењерима, планерима и менаџерима да у реалном времену приступе подацима са терена, из хангара, са аеродрома или из оперативних центара, чиме се убрзава доношење одлука и повећава оперативна флексибилност. Из тог разлога, развијено решење је примарно дизајнирано за таблет уређаје, као савремена алтернатива класичним веб апликацијама.

Мотивација за израду овог рада проистекла је из потребе за модерним, модуларним и интелигентним информационим системом који би подржао оперативно и техничко управљање комерцијалном авио-компанијом. Посебан акценат стављен је на интеграцију предикционих модела који могу унапредити планирање одржавања и процену оперативних параметара летова, чиме се повећава поузданост система и смањују оперативни трошкови.

Последњих година авио-индустрија се суочава са озбиљним поремећајима у ланцима снабдевања резервним деловима, што често доводи до значајног продужења трајања одржавања у односу на планиране рокове. Класични *AMOS* системи се углавном ослањају на статичке процене трајања радова, које не узимају у обзир актуелне глобалне околности. Интеграцијом предикционог модела машинског учења омогућава се реалистичнија процена датума завршетка одржавања на основу историјских података, чиме се унапређује планирање расположивости флоте.

Слични проблеми присутни су и у оперативном делу авио-саобраћаја. Повећање броја летова и авиона, загушење ваздушног простора, као и недостатак контролора летења, доводе до честих кашњења и одступања од планираног времена трајања лета (енгл. *block time*). Због тога је у оквиру система имплементиран предикциони модел машинског учења за процену трајања лета на основу карактеристика руте, чиме се омогућава прецизније планирање реда летења и ресурса.

Предмет овог рада је развој интелигентног *AMOS* система намењеног комерцијалној авио-компанији, који обједињује управљање флотом, рутама, одржавањем и корисницима, уз примену техника машинског учења за регресивне предикције кључних параметара. Систем је реализован као модуларна вишеслојна апликација са одвојеним корисничким интерфејсом, серверском логиком и базом података.

Циљ рада јесте пројектовање и имплементација скалабилног и безбедног *AMOS* система који омогућава:

- централизовано управљање авионима, рутама и одржавањем,
- флексибилан систем ауторизације заснован на ролама и пермисијама,
- интеграцију интелигентних компоненти за предикцију трајања летова и одржавања,
- доступност система на преносним уређајима ради подршке савременим облицима рада,
- једноставно проширивање система у складу са будућим потребама авио-компаније и трендовима авио-индустрије.

У раду ће бити приказана архитектура система, коришћене технологије, имплементациони детаљи, као и функционалности развијеног решења. Посебан део биће посвећен примени машинског учења у оквиру система, чиме се класични концепт *AMOS* система проширује интелигентним механизмима подршке одлучивању.

Рад је организован на следећи начин: у другом поглављу дат је преглед постојећих решења и сродних система из области управљања одржавањем и оперативним процесима у авио-индустрији. Треће поглавље обухвата опис коришћених технологија и архитектонских приступа примењених у реализацији система. У четвртом поглављу представљена је целокупна архитектура система и начин комуникације између појединих компоненти. Пето поглавље посвећено је имплементационим детаљима развијеног решења, док је у шестом поглављу дат приказ функционалности апликације. На самом крају, у закључку су сумирани постигнути резултати и размотрени могући правци даљег развоја система.

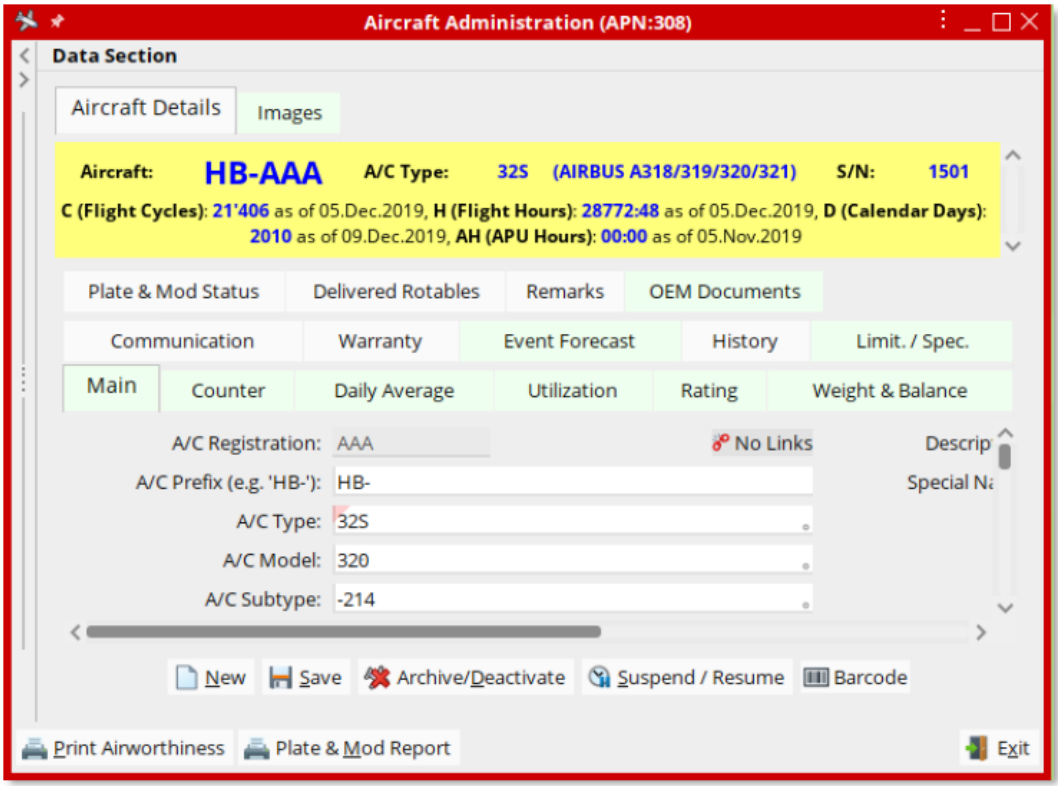
2. Преглед постојећих решења

Савремене авио-компаније користе специјализоване информационе системе за подршку оперативном и техничком управљању флотом. Ови системи, познати под акронимом *AMOS*, интегришу податке о авионима, одржавању, рутама, залихама и људским ресурсима. На тржишту постоји више комерцијалних решења која се користе у великим и средњим авио-компанијама, а у наставку су приказана нека од најзначајнијих.

2.1. AMOS (Swiss Aviation Software)

AMOS, развијен од стране компаније *Swiss Aviation Software*, представља једно од најраспрострањенијих решења у области управљања одржавањем и операцијама авио-компанија. Систем је првенствено фокусиран на технички део пословања и обухвата модуле за планирање одржавања, праћење ресурса, управљање залихама и техничку документацију.

На Слика 1 приказан је пример корисничког интерфејса *AMOS* система, који илуструје класичан десктоп приступ управљању техничким подацима о авиону.



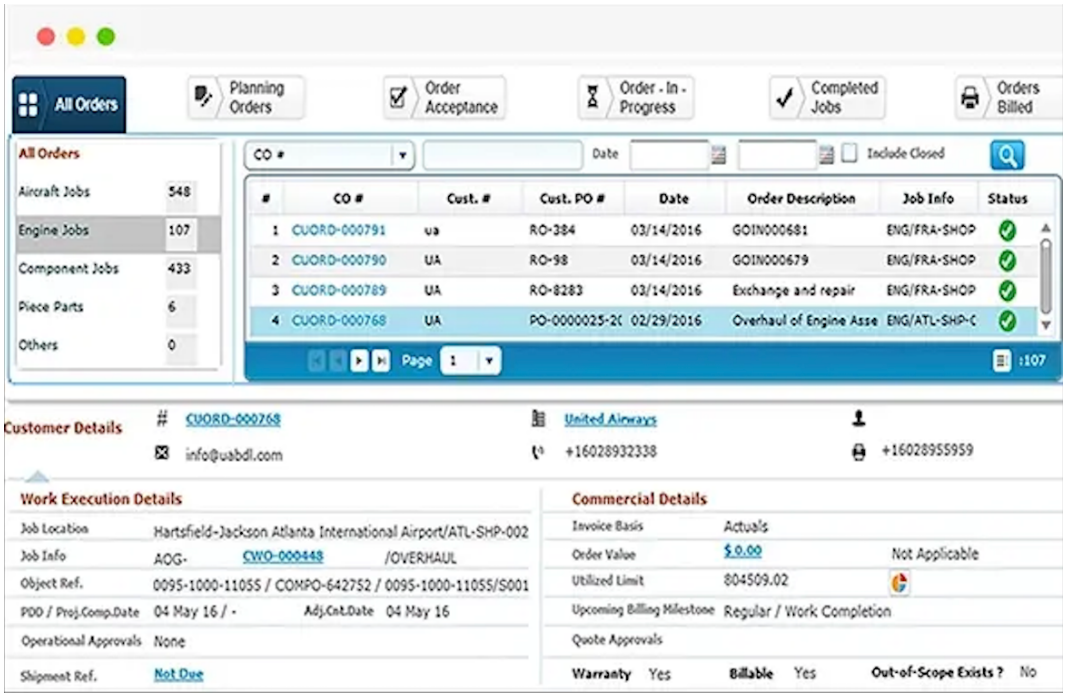
Слика 1: Пример корисничког интерфејса AMOS система - приказ техничких података о авиону

Предност овог система огледа се у његовој стабилности и широкој примени у индустрији, док се као недостатак може навести релативно ограничена флексибилност у погледу прилагођавања специфичним потребама појединих авио-компанија. Такође, класичне верзије система су примарно оријентисане на десктоп окружење, док је подршка за мобилне платформе често реализована кроз додатне модуле.

2.2. Ramco Aviation Suite

Ramco Aviation Suite представља интегрисану платформу која покрива одржавање, операције, финансије и управљање људским ресурсима. Систем је конципиран као свеобухватно решење за авио-компаније које желе да обједине више пословних функција у јединствену информациону платформу.

На Слика 2 приказан је пример корисничког интерфејса *Ramco Aviation Suite* система, који илуструје интегрисани приступ управљању техничким, оперативним и комерцијалним процесима.



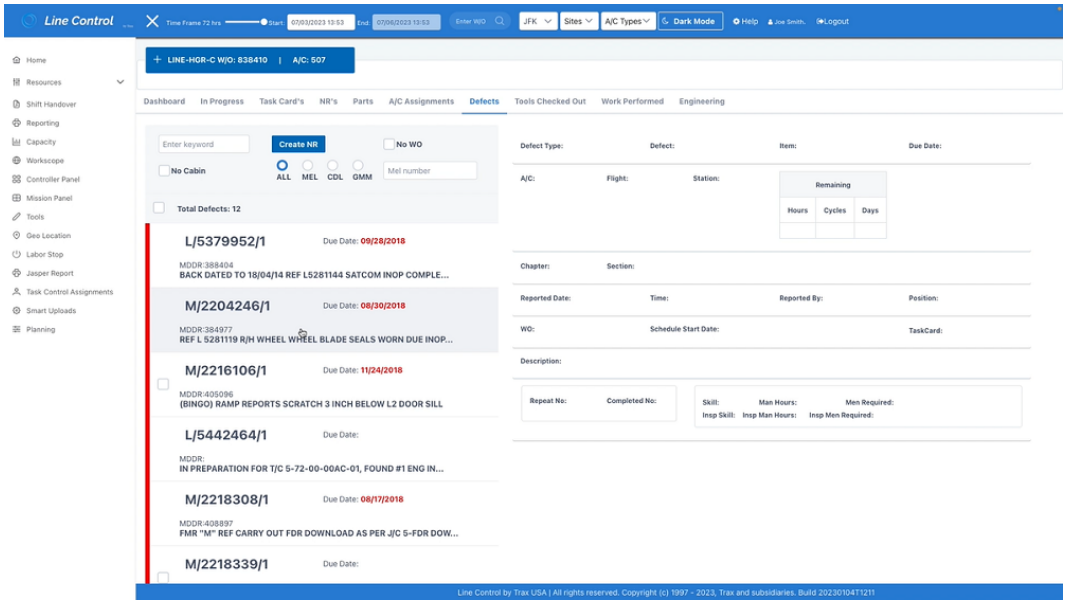
Слика 2: Пример корисничког интерфејса Ramco Aviation Suite система – интегрисани приказ радних налога и оперативних података

Једна од главних карактеристика *Ramco Aviation Suite* система јесте висока модуларност, која омогућава компанијама да имплементирају само оне компоненте које су им потребне. Међутим, због своје сложености, овакви системи често захтевају дуготрајан процес имплементације и прилагођавања, као и значајне трошкове одржавања.

2.3. Trax Maintenance

Trax Maintenance је информациони систем фокусиран првенствено на област техничког одржавања авиона. Систем омогућава праћење радних налога, техничких записа, ресурса и усклађености са регулаторним захтевима, што га чини погодним за употребу у оквиру организација са израженим техничким и регулаторним захтевима. Посебан акценат стављен је на документовање процеса одржавања и праћење животног циклуса компоненти.

На Слика 3 приказан је пример корисничког интерфејса *Trax Maintenance* система, који илуструје управљање дефектима и радним задацима у оквиру линијског одржавања авиона.



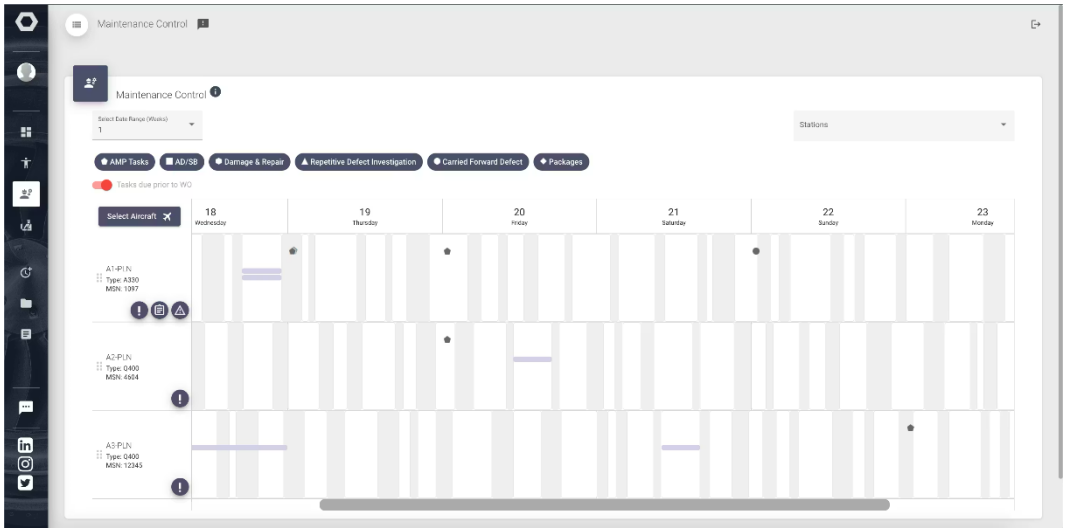
Слика 3: Trax Maintenance – приказ управљања дефектима и задацима линијског одржавања

Иако *Trax Maintenance* пружа поуздану подршку за одржавање флоте, његова функционалност је у мањој мери усмерена на оперативне апсекте, као што су управљање рутама и планирање летова. Из тог разлога, систем не покрива у потпуности све пословне процесе авио-компаније, већ се најчешће користи у комбинацији са другим системима који обухватају оперативно планирање и управљање. Оваква подела функционалности може довести до повећане сложености интеграције и потребе за синхронизацијом података између више система.

2.4. OASES (Commsoft)

OASES је модуларни *AMOS* систем који интегрише управљање одржавањем, залихама и техничком документацијом. Посебна пажња посвећена је флексибилности система и могућности прилагођавања различитим типовима оператера, укључујући и авио-компаније и организације за одржавање авиона.

На Слика 4 приказан је пример корисничког интерфејса *OASES* система, који илуструје планирање и контролу активности одржавања авиона у временском распону.



Слика 4: Пример корисничког интерфејса OASES система – планирање активности одржавања

Иако *OASES* нуди савремене функционалности, већина класичних имплементација и даље се ослања на традиционалне корисничке интерфејсе и мануелне процесе планирања, без шире примене интелигентних предикционих модела.

2.5. Компаративна анализа постојећих AMOS решења

У циљу јаснијег сагледавања позиције развијеног система у односу на постојећа комерцијална *AMOS* решења, извршена је компаративна анализа анализираних система на основу заједничких функционалних и архитектонских карактеристика. Приликом поређења нису коришћени квантитативни показатељи, већ је фокус стављен на квалитативну подршку кључним аспектима савремених *AMOS* система, као што су мобилна доступност, модуларност, флексибилност архитектуре, подршка оперативним и техничким процесима, као и примена интелигентних предикционих механизама.

За сваку карактеристику коришћена су три нивоа подржаности – не подржава, делимично подржава и потпуно подржава. Оваква класификација омогућава једноставан и прегледан увид у функционалне разлике између система, као и уочавање области у којима поједина решења имају предност. Резултати анализе приказани су у Табела 1, која омогућава директно поређење комерцијалних *AMOS* система са развијеним интелигентним *AMOS* системом.

Табела 1: Компаративна анализа AMOS система

Карактеристика	*AMOS*	*Ramco Aviation Suite*	*Trax Maintenance*	*OASES*	Развијени систем
Управљање техничким одржавањем	Потпуно подржава	Потпуно подржава	Потпуно подржава	Потпуно подржава	Потпуно подржава
Управљање оперативним процесима	Делимично подржава	Потпуно подржава	Не подржава	Делимично подржава	Потпуно подржава
Модуларна архитектура	Делимично подржава	Потпуно подржава	Делимично подржава	Потпуно подржава	Потпуно подржава
Мобилна / таблет подршка	Делимично подржава	Делимично подржава	Делимично подржава	Делимично подржава	Потпуно подржава
Интелигентне предикције	Не подржава	Делимично подржава	Не подржава	Не подржава	Потпуно подржава
Permission-based контрола приступа	Делимично подржава	Потпуно подржава	Делимично подржава	Делимично подржава	Потпуно подржава
Архитектура заснована на сервисима	Делимично подржава	Делимично подржава	Делимично подржава	Делимично подржава	Потпуно подржава
Фокус на мобилном коришћењу	Не подржава	Делимично подржава	Не подржава	Не подржава	Потпуно подржава

На основу приказане анализе може се уочити да развијени систем у оквиру овог рада уводи јасне предности у погледу мобилне доступности, флексибилне контроле приступа и примене интелигентних механизама подршке одлучивању, чиме се издваја у односу на анализирана комерцијална решења.

2.6. Поређење са развијеним системом

Док већина комерцијалних *AMOS* решења представља комплексне и свеобухватне системе, често оријентисане ка великим авио-компанијама и дуготрајним процесима имплементације, развијени систем у оквиру овог рада пројектован је са другачијим приоритетима. Акценат је стављен на једноставност, модуларност и могућност постепеног проширивања функционалности у складу са реалним потребама корисника.

Посебна пажња посвећена је мобилној доступности система, која омогућава употребу апликације у оперативним условима, као што су аеродроми, хангари и контролни центри. За разлику од анализираних комерцијалних решења, која мобилну подршку често реализују као додатни модул, развијени систем је од самог почетка дизајниран као мобилно оријентисана апликација.

Додатна вредност предложеног решења огледа се у интеграцији интелигентних компоненти заснованих на машинском учењу, које омогућавају предикцију трајања летова и одржавања. На тај начин, класичан *AMOS* концепт проширен је механизмима подршке одлучивању, што је од посебног значаја у условима савремене авио-индустрије, коју карактеришу повећан обим саобраћаја и изазови у планирању ресурса.

Коначно, имплементација *permission-based* модела контроле приступа омогућава фино подешавање овлашћења корисника и безбедан приступ функционалностима система. Овакав приступ олакшава примену система у реалном пословном окружењу и представља добру основу за његово даље проширивање и интеграцију са другим информационим системима.

3. Коришћене технологије

Развијени интелигентни *AMOS* систем реализован је као вишеслојна апликација која се састоји од корисничког интерфејса, серверске логике и базе података. Приликом избора технологије, посебан акценат стављен је на модуларност, скалабилност, безбедност и могућност интеграције интелигентних компоненти. У наставку су описане кључне технологије коришћене у реализацији система.

3.1. Flutter (програмски језик Dart)

Flutter представља савремени радни оквир програмског језика *Dart*, развијен од стране компаније *Google* и први пут објављен у мају 2017. године, намењен развоју мултиплатформских апликација за *Android*, *iOS*, *Windows*, *macOS*, *Linux* и веб платформе коришћењем јединствене базе кода. У оквиру овог пројекта *Flutter* је коришћен за развој корисничког интерфејса, примарно оптимизованог за таблет уређаје.

Предност *Flutter*-а огледа се у високој перформанси захваљујући директном исцртавању корисничког интерфејса помоћу графичког механизма *Skia*, као и у брзом развоју апликација путем механизма *hot reload*. Коришћењем *Flutter*-а омогућено је конзистентно корисничко искуство на различитим платформама, уз смањење трошкова развоја и одржавања апликација.

Избор *Flutter*-а додатно је оправдан потребом за мобилношћу система, јер омогућава приступ апликацији ван класичног канцеларијског окружења, што је посебно значајно у савременим условима рада на даљину.

3.2. ASP.NET (програмски језик C#)

Серверска страна система имплементирана је коришћењем радног оквира *ASP.NET* програмског језика *C#*, развијен од стране компаније *Microsoft* и први пут објављен у јануару 2002. године. Овај радни оквир представља робусну платформу за развој веб сервиса и дистрибуираних апликација.

ASP.NET омогућава изградњу скалабилних *REST* сервиса који обрађују захтеве клијентске апликације и комуницирају са базом података. У оквиру пројекта коришћен је за имплементацију пословне логике система и интеграцију са интелигентним компонентама за машинско учење.

Предности *ASP.NET* платформе огледају се у високој сигурности, доброј подршци за аутентификацију и ауторизацију, као и у једноставној интеграцији са осталим технологијама развијене од стране компаније *Microsoft*. Коришћењем овог оквира омогућено је јасно раздвајање пословне логике од корисничког интерфејса, чиме се постиже боља одрживост система.

3.3. ML.NET (програмски језик C#)

ML.NET представља *Microsoft*-ов радни оквир, први пут објављен у мају 2018. године, намењен примени алгоритама машинског учења у *.NET* окружењу. У оквиру овог пројекта *ML.NET* коришћен је за развој регресионих модела који омогућавају предикцију трајања летова и процену датума завршетка одржавања.

Предност *ML.NET* огледа се у једноставној интеграцији са *ASP.NET* апликацијама, чиме се омогућава имплементација интелигентних функционалности без потребе за коришћењем екстерних сервиса. Тиме се постиже већа контрола над подацима и смањује зависност од трећих страна, што повећава поузданост целокупног система.

3.4. Microsoft SQL Server

За складиштење података коришћен је *Microsoft SQL Server*, први пут објављен у априлу 1989. године, који представља релациони систем за управљање базама података намењен раду са великим количинама структурисаних података. База података садржи информације о авионима, типовима авиона, рутама, одржавањима, корисницима и ролама у систему.

SQL Server омогућава високу поузданост, интегритет података и подршку за сложене упите, што је од кључне важности за информациони систем у области авио-индустрије. Такође, омогућена је једноставна интеграција са *ASP.NET* апликацијама путем објектно-релационих мапера као што је *Entity Framework Core* (*EF Core*).

3.5. JWT аутентификација и ауторизација

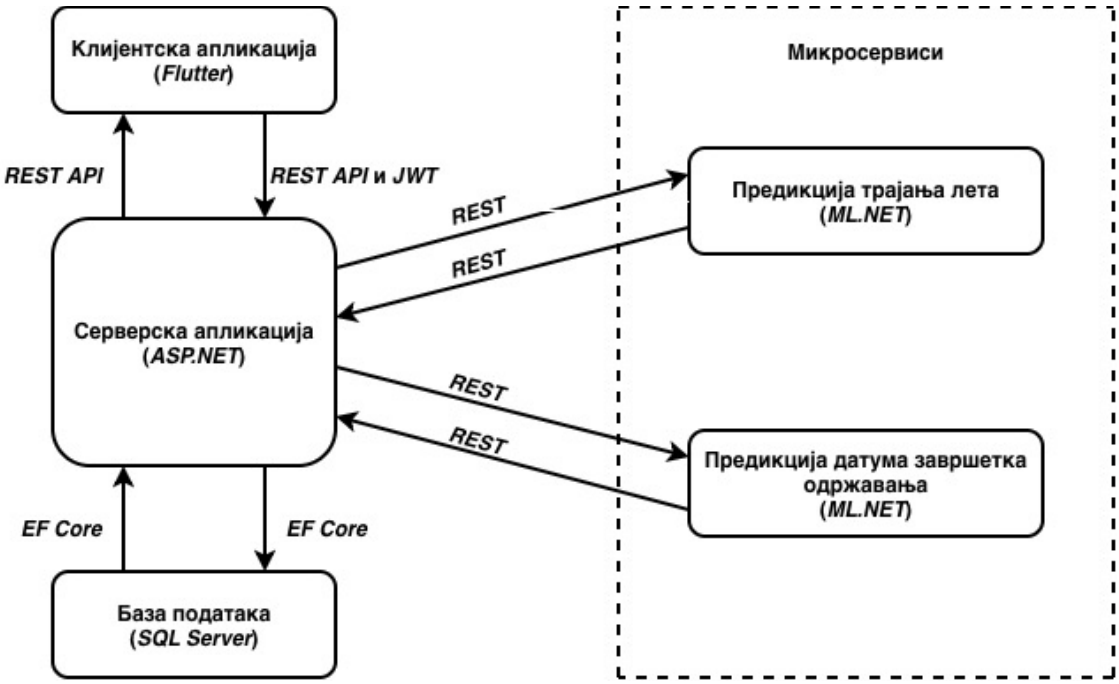
За реализацију безбедног приступа систему коришћен је механизам аутентификације заснован на токенима (*JSON Web Token - JWT*). Након успешне пријаве, кориснику се додељује токен који се користи за ауторизацију свих наредних захтева према серверској апликацији.

Овакав приступ омогућава развој дистрибуираног система у којем клијентска апликација и серверска страна функционишу независно, уз очување високог нивоа безбедности. Аутентификација базирана на токенима је посебно погодна за мобилне апликације и скалабилне *cloud* системе.

4. Архитектура система

Развијени интелигентни *AMOS* систем пројектован је као дистрибуирана сервисно-оријентисана вишеслојна апликација, у којој су кориснички интерфејс, серверска логика и база података логички и физички раздвојени. Оваква архитектура омогућава високу скалабилност, лакше одржавање и једноставну надоградњу система у будућности, као и независан развој и распоређивање појединих сервиса.

На Слика 5 приказана је укупна архитектура система и међусобна комуникација између клијентске апликације, серверског слоја, базе података и независних микросервиса.



Слика 5: Архитектура развијеног интелигентног AMOS система и ток комуникације између компоненти

Систем је организован према класичној тролојној архитектури, као што је приказано на Слика 5, која се састоји од:

- презентационог слоја (*frontend*),
- апликативног слога (*backend*),
- слоја података (*database*).

Поред наведених слојева, у оквиру серверске стране имплементиран је и засебан слој интелигентних компоненти заснованих на машинском учењу, реализован у виду независних сервиса, чиме се класична слојевита архитектура проширује сервисно-оријентисаним приступом. Оваква комбинација архитектонских приступа омогућава јасну организацију система, уз задржавање флексибилности у распоређивању појединих компоненти.

4.1. Презентациони слој (frontend)

Презентациони слој реализован је коришћењем *Flutter* радног оквира и представља кориснички интерфејс система. Овај слој је одговоран за интеракцију са корисником, приказ података и слање захтева ка серверској апликацији.

Frontend апликација је оптимизована за таблет уређаје, што омогућава употребу система у покрету, у хангарима, на аеродромима и у оперативним центрима. Кориснички интерфејс је реализован у виду модуларног менија, при чему се доступне функционалности динамички прилагођавају роли и пермисијама улогованог корисника.

Сва комуникација са серверском страном реализована је путем *HTTP* протокола, коришћењем *REST API* интерфејса. Клијентска страна не приступа директно бази података, чиме се обезбеђује јасно раздвајање одговорности између слојева.

4.2. Апликативни слој (backend)

Апликативни слој имплементиран је коришћењем *ASP.NET* радног оквира и представља централну компоненту система. Овај слој је одговоран за обраду свих захтева клијентске апликације, примену пословне логике, управљање безбедношћу и комуникацију са базом података.

Backend је реализован у складу са *Model-Controller* архитектурним приступом, где:

- модели представљају доменске ентитете система (авиони, руте, одржавања, корисници, роле),
- контролери обрађују *HTTP* захтеве и враћају одговарајуће одговоре клијенту.

Оваква организација омогућава бољу прегледност програмског кода, једноставније тестирање и лакше проширивање функционалности система.

4.3. Слој података (database)

Слој података реализован је коришћењем *Microsoft SQL Server* система за управљање базама података. База података представља централно складиште свих релевантних информација система.

У систему је имплементирана релациона база података која садржи осам основних табела:

- авиони (*Aircraft*),
- типови авиона (*AircraftTypes*),
- аеродроми (*Airports*),
- одржавања (*Maintenance*),
- руте (*Routes*),
- детаљи рута (*RouteDetail*),
- корисници (*Users*),
- роле (*Roles*).

Оваква структура омогућава јасно моделовање односа између ентитета и ефикасно извршавање упита неопходних за рад апликације.

4.4. Слој интелигентних компоненти

Слој интелигентних компоненти у развијеном систему реализован је као засебна сервисна целина, у складу са принципима сервисно-оријентисане архитектуре. За разлику од монолитних решења, у којима су модели машинског учења директно уграђени у основну серверску апликацију, у овом систему су интелигентне компоненте имплементиране као две независне *provider* апликације.

Свака *provider* апликација представља самосталан сервис који извршава специфичну функцију машинског учења:

- сервис за предикцију трајања лета (*block time*),
- сервис за предикцију датума завршетка одржавања.

Ови сервиси реализовани су коришћењем *ML.NET* радног оквира и функционишу независно од главне серверске (*API*) апликације. Главни *backend* систем остварује комуникацију са *provider* сервисима путем дефинисаних програмских интерфејса, при чему се функционалности машинског учења користе на захтев, без директне зависности од имплементационих детаља.

Оваква архитектура доноси више предности:

- омогућава независно одржавање и надоградњу модела машинског учења,
- смањује сложеност главне серверске апликације,
- повећава поузданост система, јер отказ једног сервиса не мора довести до потпуног прекида рада система,
- омогућава лаку замену или проширење интелигентних компоненти у будућности.

На овај начин, класичан *AMOS* систем проширен је интелигентним микросервисима, чиме се постиже већи степен флексибилности и скалабилности целокупне архитектуре.

4.5. Ток комуникације у систему

Комуникација између слојева развијеног интелигентног *AMOS* система одвија се према јасно дефинисаном и контролисаном току, који обезбеђује раздвајање одговорности, безбедност и поузданост рада апликације. Целокупна интеракција корисника са системом реализује се путем презентационог слоја, док се обрада података и пословна логика извршавају на страни сервера.

Ток комуникације може се описати кроз следеће кораке:

- корисник иницира захтев путем *Flutter* клијентске апликације,
- захтев се шаље *backend* серверу путем *REST API* позива,
- сервер врши аутентификацију и ауторизацију корисника помоћу *JWT* токена,
- након успешне верификације, примењује се пословна логика и, по потреби, остварује се комуникација са *provider* сервисима машинског учења ради добијања предикције,
- серверска апликација приступа бази података и обрађује релевантне податке,
- обрађени подаци враћају се клијентској апликацији ради приказа кориснику.

Оваква организација комуникације омогућава да клијентска апликација не поседује директан приступ подацима, већ да сваки захтев пролази кроз серверски слој који контролише приступ и спроводи безбедносне механизме. На тај начин обезбеђен је висок степен заштите података, као и могућност централизованог управљања пословним правилима система.

Описани ток комуникације између појединих слојева система графички је илустрован на Слика 5, кроз приказ међусобних веза и праваца размене података између клијентске апликације, серверског слоја, базе података и независних интелигентних сервиса.

4.6. Безбедносни и скалабилни аспекти архитектуре

Поред функционалних захтева, приликом пројектовања архитектуре система посебна пажња посвећена је нефункционалним захтевима, као што су безбедност, скалабилност и поузданост.

Безбедност система обезбеђена је вишеслојним приступом. Корисничка аутентификација реализована је на серверској страни применом *JWT* (*JSON Web Token*) механизма, чиме се спречава неовлашћен приступ ресурсима система. Сва комуникација између клијентске и серверске апликације одвија се путем сигурних *HTTP* захтева, при чему клијентска страна нема директан приступ бази података. Додатно, *permission-based* модел ауторизације омогућава прецизну контролу приступа функционалностима у складу са ролом корисника.

Скалабилност система постигнута је раздвајањем појединих компоненти у независне сервисе. Посебно значајан аспект представља издвајање интелигентних компоненти у засебне *provider* апликације, што омогућава њихово независно оптерећивање и надоградњу без утицаја на основи *API* систем. Оваква архитектура омогућава хоризонтално скалирање сервиса у складу са растом броја корисника и количине података.

Поузданост система повећана је раздвајањем критичних компоненти, чиме се смањује утицај евентуалних отказа појединих сервиса на рад целокупне апликације. На овај начин систем је припремљен за примену у реалном продукционом окружењу авио-компаније.

5. Имплементациони детаљи

У овом поглављу описана је реализација кључних компоненти система, са посебним освртом на механизме аутентификације и ауторизације, организацију података у бази, као и интеграцију интелигентних сервиса заснованих на машинском учењу.

5.1. Аутентификација и ауторизација корисника

Систем користи механизам аутентификације заснован на *JWT* (*JSON Web Token*) технологији. Приликом покретања апликације, корисник је дужан да се пријави уносом корисничког имена и лозинке. Клијентска апликација шаље унете податке серверској апликацији, која врши валидацију унетих креденцијала.

У случају успешне пријаве, сервер генерише *JWT* токен који садржи идентификационе податке о кориснику, његову улогу и припадајуће пермисије, као што илуструје Програмски код 1. Овај токен се враћа клијентској апликацији и користи се за ауторизацију свих наредних захтева.

```
public class Auth
{
    public Auth(string token, List<string> permissions, User user)
    {
        this.Token = token;
        this.Permissions = permissions;
        this.User = user;
    }

    public User User { get; set; }

    public string Token { get; set; }

    public List<string> Permissions { get; set; } = [];

    public DateTime LastActivity { get; set; } = DateTime.Now;

    public bool Allowed(string permission)
    {
        return Permissions.Contains(permission);
    }
}
```

Програмски код 1: Имплементација JWT објекта

Токен се прослеђује у *HTTP* заглављу сваког захтева, чиме сервер у сваком тренутку може да верификује идентитет корисника. Оваквим приступом елиминише се потреба за чувањем сесија на серверској страни, што доприноси скалабилности система.

5.2. Permission-based модел контроле приступа

Поред основне аутентификације, у систему је имплементиран флексибилан механизам ауторизације заснован на ролама и пермисијама. Сваки корисник у систему има додељену улогу, као и додатне дозвољене и забрањене пермисије, као што илуструје Програмски код 2.

Роле дефинишу основни скуп пермисија, док се индивидуалне пермисије користе за фино подешавање приступа. На тај начин омогућено је да два корисника са истом улогом имају различите нивое приступа функционалностима система.

```
public class Role
{
    public int Id { get; set; }

    public string? Name { get; set; }

    public string? AllowedFunctions { get; set; }
}

public class User
{
    public int Id { get; set; }

    public string? FirstName { get; set; }

    public string? LastName { get; set; }

    public string? UserName { get; set; }

    public string? Password { get; set; }

    public Role? Role { get; set; }

    public string? AllowedFunctions { get; set; }

    public string? ForbiddenFunctions { get; set; }
}
```

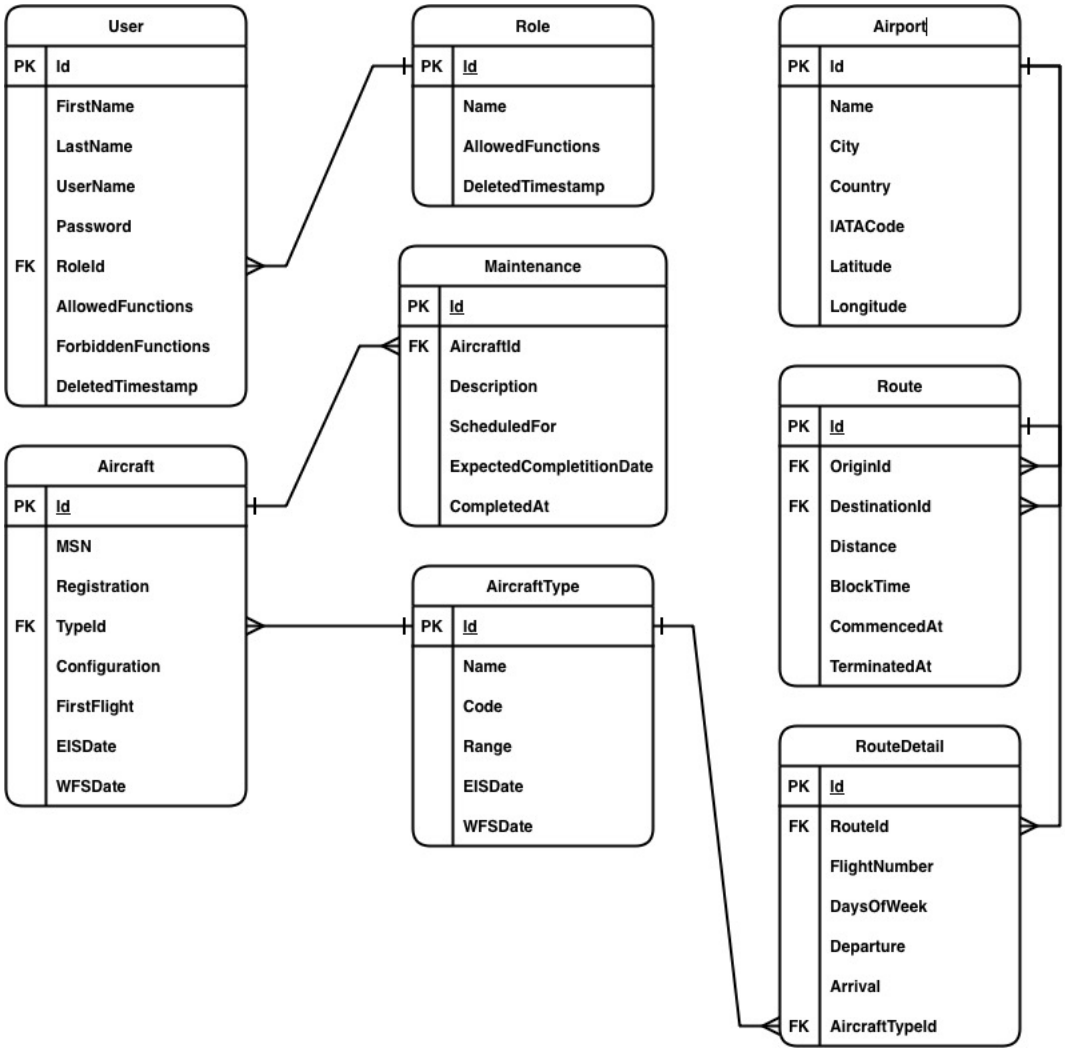
Програмски код 2: Имплементација објеката рола и профила

Приликом сваког захтева, серверска апликација проверава да ли корисник поседује одговарајућу пермисију за извршење тражене операције. Оваква имплементација омогућава лако проширивање система додавањем нових функционалности без нарушавања постојеће безбедносне логике.

5.3. Организација података у бази

База података је пројектована у складу са релационим моделом и представља централно складиште свих информација система. Основне табеле у систему, приказане на Слика 6, су:

- *Aircraft* – подаци о појединачним авионима,
- *AircraftTypes* – подаци о подржаним типовима авиона,
- *Airports* – подаци о подржаним аеродромима,
- *Routes* и *RouteDetail* – подаци о рутама и детаљима руте,
- *Maintenance* – подаци о одржавањима авиона,
- *Users* – подаци о регистрованим корисницима,
- *Roles* – подаци о подржаним ролама.



Слика 6: Релациона шема базе података

Табеле су повезане као што је приказано на Слика 6, док су релације између табела дефинисане страним кључевима (енгл. *foreign key*), чиме је обезбеђен интегритет података. Оваква структура омогућава ефикасно извршавање упита неопходних за приказ комплексних пословних информација у оквиру апликације.

5.4. Чување лозинки и безбедност података

Из безбедносних разлога, лозинке корисника не чувају се у оригиналном облику у бази података. Приликом регистрације или промене лозинке, над лозинком се спроводи криптографски *hash* алгоритам, као што је илустровано на Слика 7, након чега се у бази чува искључиво заштићена вредност. Приликом логовања у систем, корисник уноси оригиналну лозинку, над којом се затим спроводи идентична *hash* функције ради упоређивања са подацима у бази.

```
оригинална лозинка: Password123!
лозинка сачувана у бази података:
a2eb092319924ebea53af947ae112be8a9748d3db1ee0c20cc0a255b4e723245a2eb092319924eb
ea53af947ae112be8a9748d3db1ee0c20cc0a255b4e723245
```

Слика 7: Пример примене hash функције над унетом лозинком

Оваквим приступом спречава се компромитовање корисничких налога у случају неовлашћеног приступа бази података, што је од изузетне важности за информациони систем у области авио-индустрије.

5.5. Интеграција provider сервиса машинског учења

Интелигентне компоненте система реализоване су као два независна *provider* сервиса. Сваки сервис извршава специфичну предиктивну функцију и комуницира са главном *API* апликацијом путем дефинисаних *REST* интерфејса.

Када корисник уноси нову руту или будуће одржавање, серверска апликација прослеђује релевантне податке одговарајућем *provider* сервису путем јавних метода које сервис експонира ка својим корисницима, као што илуструје Програмски код 3. Сервис обрађује податке коришћењем обученог модела и враћа процену која се користи у процесу доношења одлука.

```
public float Predict(RouteData input)
{
    var prediction = engine.Predict(input);

    return prediction.Score;
}

public (double rmse, double r2) Evaluate(int testSetSize)
{
    var testData = DataGenerator.GenerateTestSet(testSetSize);
    var testDataView = mlContext.Data.LoadFromEnumerable(testData);
    var predictions = model.Transform(testDataView);
    var metrics = mlContext.Regression.Evaluate(predictions);

    return (metrics.RootMeanSquaredError, metrics.RSquared);
}
```

Програмски код 3: Имплементација метода које сервис експонира својим корисницима

5.6. Тренирање модела машинског учења

Приликом покретања серверске апликације и предикционих сервиса врши се учитавање и тренирање модела на основу историјских података из базе. На тај начин обезбеђује се да модел користи актуелне податке за генерисање предикција.

Поред тренирања модела на основу историјских података, у систему је имплементиран механизам резервног учитавања модела. На страни главне *API* апликације чувају се претходно генерисани модели у виду *.zip* фајлова, који представљају снимак последњег валидног обученог модела. Приликом покретања система, уколико тренирање није планирано или уколико дође до грешке током процеса обучавања (нпр. недоступност базе, недовољно података или насилни прекид), модел се учитава из одговарајућег *.zip* фајла. На тај начин обезбеђује се континуитет рада система и доступност предикција чак и условима непланираних проблема који захтевају рестартовање серверске апликације.

У продукционом окружењу, где се серверске апликације врло ретко рестартују, процес обучавања модела би било пожељно реализовати путем периодичних позадинских процеса (*background jobs*), како би се модели редовно ажурирали без прекида рада система, док би се у случају рестартовања система иницијални модел могао учитати из одговарајућег *.zip* фајла. Једна од могућих имплементација позадинских процеса би могао бити *Hangfire*.

6. Функционалности система

Развијени интелигентни *AMOS* систем обухвата више функционалних целина које покривају кључне аспекте оперативног и техничког управљања комерцијалном авио-компанијом. Функционалности система организоване су у виду модула, при чему је приступ појединим модулима условљен ролом и пермисијама пријављеног корисника.

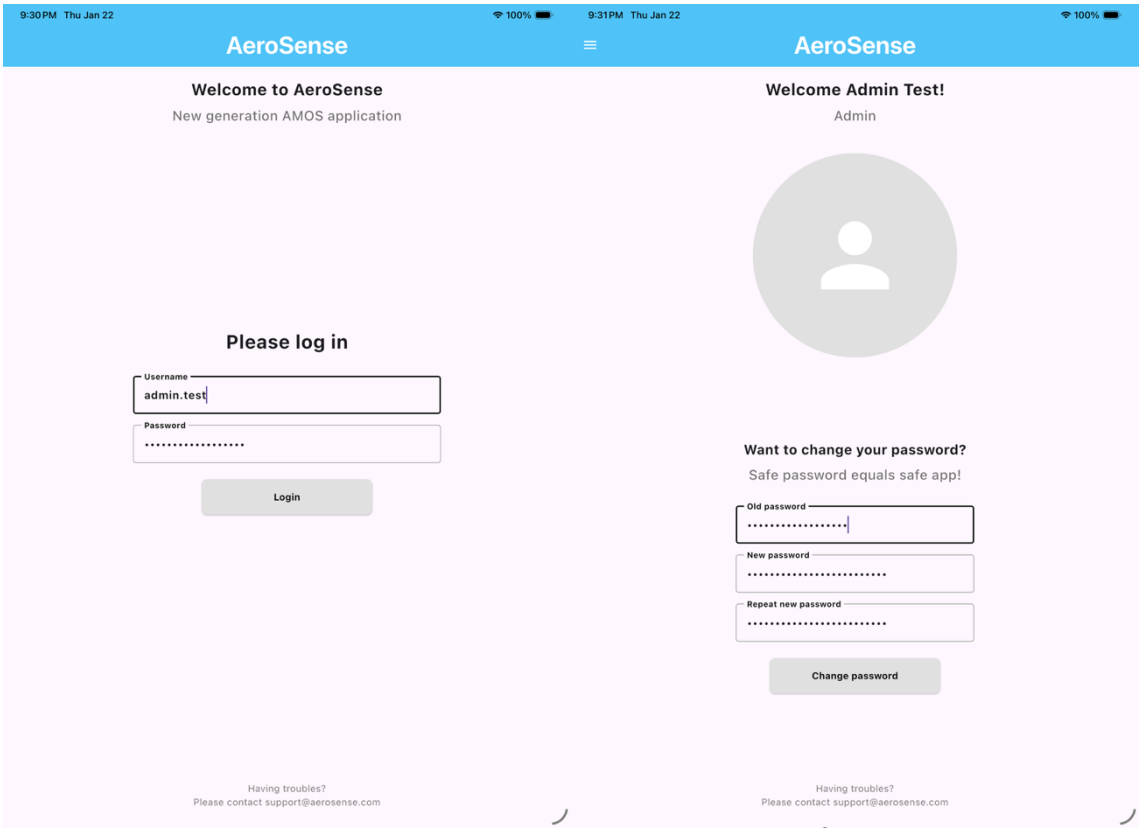
У наставку су описане основне функционалности система, груписане по логичким целинама.

6.1. Аутентификација и кориснички профил

Приликом покретања апликације, кориснику је приказана форма за пријаву, где је неопходно унети корисничко име и лозинку. Након успешне аутентификације, корисник се преусмерава на почетну страницу система, као што се може видети на Слика 8.

У случају уноса неисправних корисничких креденцијала, систем кориснику приказује одговарајућу поруку у виду кратке нотификације (енгл. *snackbar*) црвене боје. Оваквим приступом обезбеђује се јасна и ненаметљива комуникација са корисником, без откривања осетљивих информација о разлозима неуспеле пријаве.

Идентични механизам комуникације са корисником у случају грешака се користи и у осталим деловима апликације у циљу лакшег идентификовања узрока грешке.



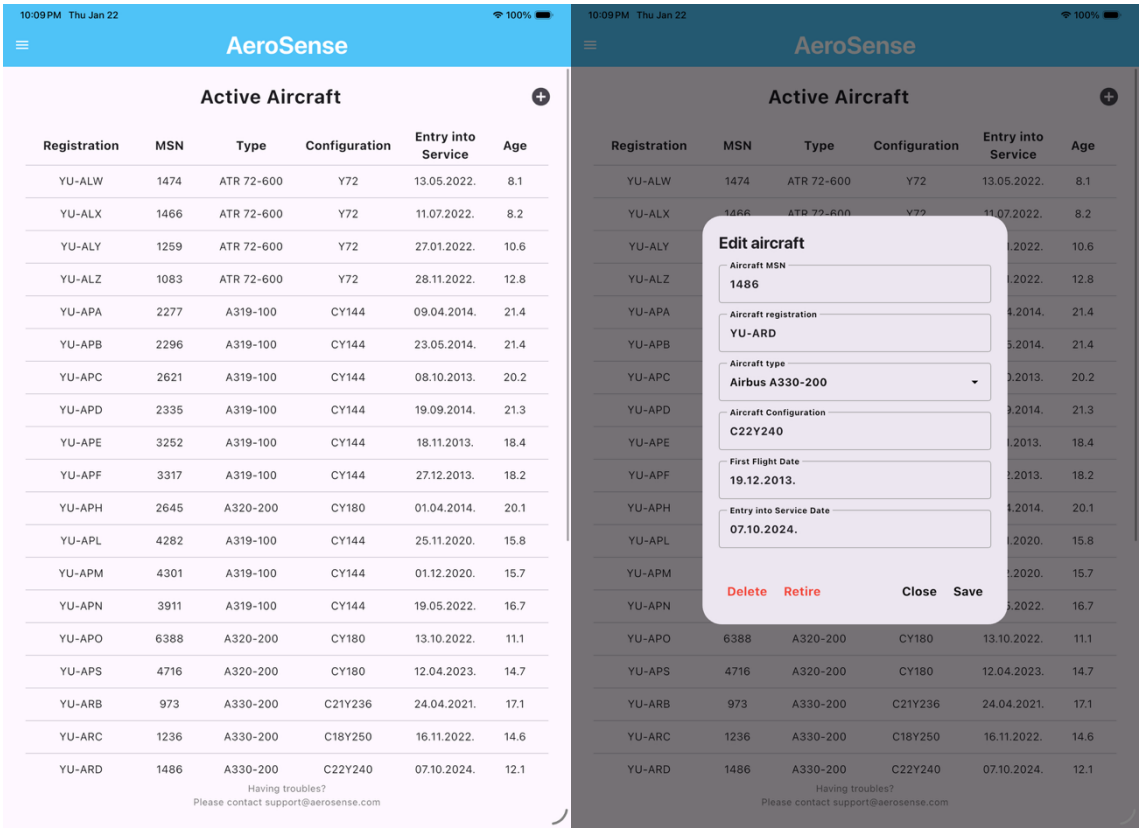
Слика 8: Екран за пријаву корисника (лево) и кориснички профил након успешног логовања (десно)

Почетна страница (профил) омогућава приказ основних података о пријављеном кориснику, као и промену лозинке. На овај начин обезбеђена је основна персонализација система и безбедно управљање корисничким налогом.

6.2. Управљање авионима

Модул за управљање авионима омогућава преглед и администрацију флоте авио-компаније и представља једну од основних функционалних целина система. Авиони су у систему класификовани као активни или пензионисани, при чему је начин приказа података у оба режима у великој мери идентичан, док се разлике огледају у доступним операцијама и приказаним атрибутима.

За активне авионе омогућено је уређивање података и покретање процеса пензионисања, као што је приказано на Слика 9, док су подаци о пензионисаним авионима доступни искључиво у *read-only* режиму, чиме се обезбеђује интегритет историјских података.



Слика 9: табеларни приказ активних авиона (лево) и едитор за измену података о авиону (десно)

У табеларном приказу активних авиона приказују се кључни идентификациони и оперативни подаци, као што су регистрација авиона, серијски број произвођача (*MSN*), тип и конфигурација летелице, датум почетка оперативне употребе у оквиру компаније, као и тренутна старост авиона. Оваква структура података омогућава брз увид у стање флоте и доношење оперативних одлука заснованих на релевантним информацијама.

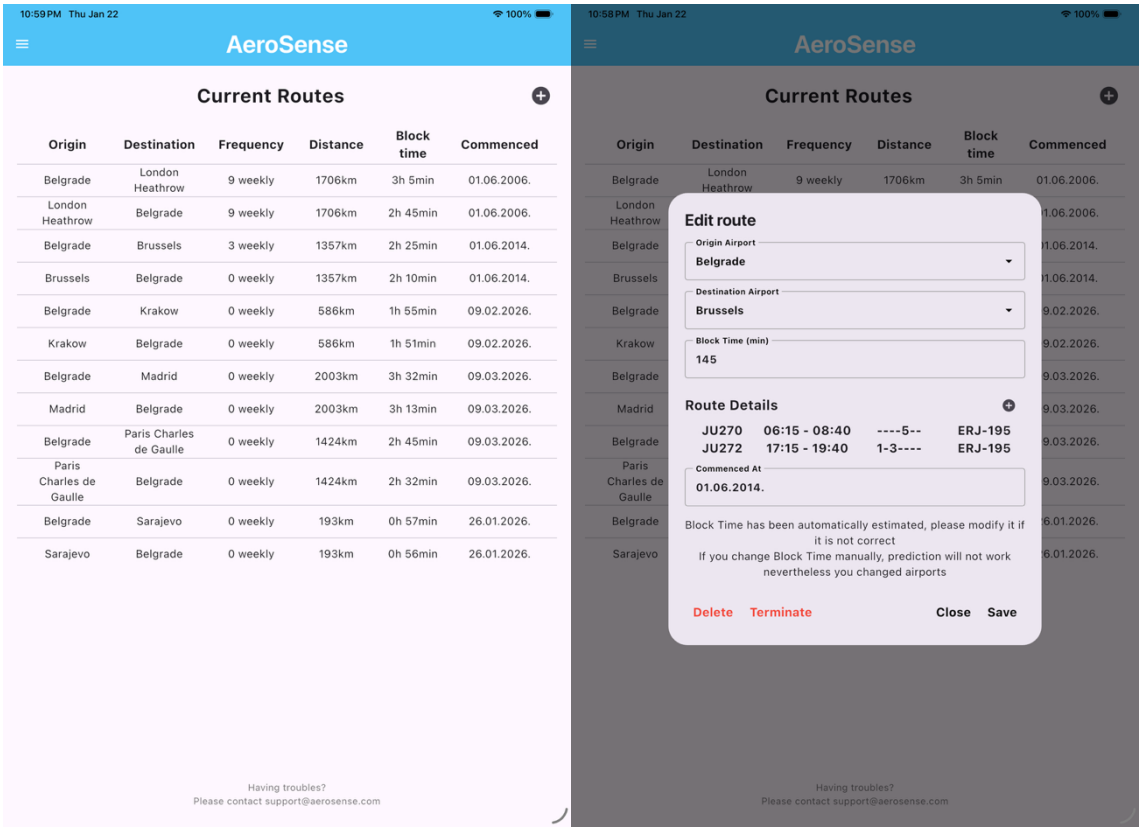
За пензионисане авионе приказује се сличан скуп података, при чему се, уместо тренутне старости и конфигурације, истиче датум завршетка оперативне употребе у компанији. На овај начин обезбеђен је прецизан историјски преглед животног циклуса сваке летелице.

Корисници са одговарајућом пермисијом могу приступити едитору за измену података о активним авионима или додавање новог авиона у флоту. Поред измена постојећих података, омогућено је и пензионисање авиона. У случају да се ради о последњем активном авиону одређеног типа, систем аутоматски пензионише и тај тип авиона, чиме се обезбеђује конзистентност података у систему.

6.3. Управљање рутама

Модул за управљање рутама омогућава евиденцију, анализу и администрацију летачке мреже авио-компаније. Руте су у систему класификоване као активне и историјске, при чему је начин приказа података у оба режима у великој мери идентичан, док се разлике огледају у доступним операцијама над подацима.

За активне руте омогућено је уређивање података и покретање процеса терминације руте, као што је приказано на Слика 10, док су историјске руте доступне искључиво у *read-only* режиму, чиме се обезбеђује очување историјских оперативних података.



Слика 10: Табеларни приказ активних рута (лево) и едитор за управљање рутом (десно)

У табеларном приказу активних рута приказују се основни оперативни подаци, као што су полазни и долазни аеродром, учесталост летења, дистанца, планирано време трајања, као и датум почетка летова. Оваква организација података омогућава кориснику брз увид у структуру летачке мреже и лаку идентификацију појединачних рута.

Едитор руте омогућава кориснику са одговарајућом пермисијом измену основних параметара руте, као и управљање детаљима руте кроз угњеждени едитор, у којем се дефинишу фреквенције летења. На овај начин омогућено је флексибилно моделирање различитих оперативних сценарија у складу са потребама компаније.

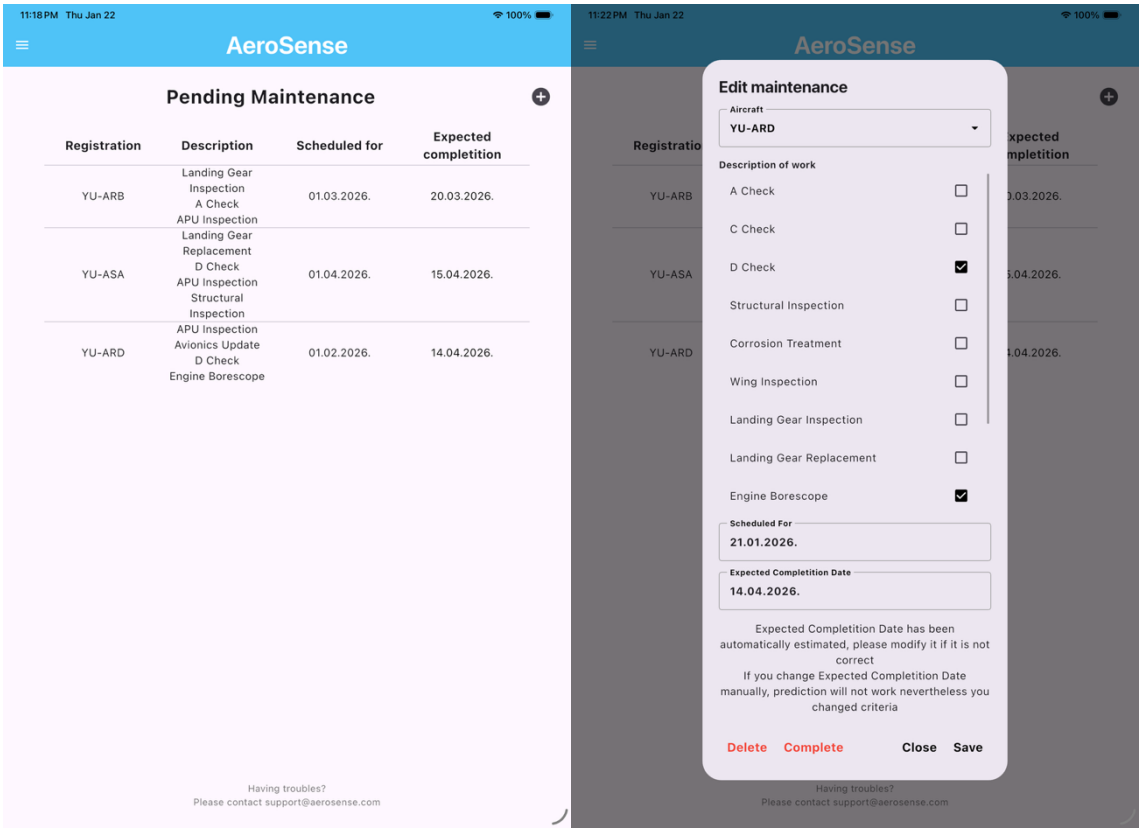
Приликом уноса нове руте или измене постојеће, систем користи интелигентни *provider* сервис за процену очекиваног трајања лета (енгл. *block time*). Предикција се заснива на историјским подацима и релевантним карактеристикама руте и служи као помоћ при доношењу оперативних одлука.

У случају терминације руте, подаци о рути се аутоматски премештају у историјски приказ, чиме се обезбеђује потпуна транспарентност промена и могућност накнадне анализе.

6.4. Управљање одржавањима

Модул за управљање одржавањима омогућава планирање, праћење и евиденцију техничких активности над авионима и представља једну од кључних функционалности *AMOS* система. Одржавања су у систему класификована као будућа и прошла, при чему је начин приказа података у оба режима у великој мери идентичан, док се разлике огледају у доступним операцијама над подацима.

За будућа одржавања омогућено је уређивање података и означавање одржавања као завршеног, као што је приказано на Слика 11, док су подаци о завршеним одржавањима доступни искључиво у *read-only* режиму, чиме се обезбеђује интегритет историјских података.



Слика 11: табеларни приказ будућих одржавања (лево) и едитор за управљање одржавањем (десно)

У табеларном приказу будућих одржавања приказују се основни подаци, као што су авион на који се одржавање односи, опис захтеваних радова, планирани датум почетка, као и процењени датум завршетка радова. Оваква организација омогућава кориснику јасан увид у предстојеће техничке активности и њихов утицај на оперативну доступност флоте.

Едитор одржавања, доступан искључиво кориснику са одговарајућом пермисијом, омогућава измену параметара одржавања, као и означавање радова као завршених. Приликом уноса новог одржавања или измене података постојећег, одржавања систем користи интелигентни *provider* сервис за процену очекиваног датума завршетка радова. Предикција се заснива на историјским подацима о сличним одржавањима и релевантним карактеристикама летелице.

По означавању одржавања као завршеног, подаци се аутоматски премештају у историјски приказ, чиме се обезбеђује прецизна евиденција и могућност анализе трајања и ефикасности извршених радова.

6.5. Административне функционалности

Административне функционалности система груписане су у оквиру *Settings* модула и представљају централно место за конфигурацију и управљање основним системским подацима на којима се базира свакодневна употреба апликације. Ове функционалности су доступне корисницима са највишим степеном привилегија, будући да неправилан или неовлашћен приступ овом делу система може довести до озбиљних безбедносних и оперативних последица.

У оквиру *Settings* модула доступно је пет основних целина:

- *Users* – управљање корисничким налозима и индивидуалним пермисијама,
- *Roles* – управљање ролама и глобалним пермисијама,
- *Active Aircraft Types* – управљање активним типовима авиона,
- *Historic Aircraft Types* – преглед историјских типова авиона,
- *Airports* – управљање подржаним аеродромима у систему.

Секција за управљање корисницима омогућава креирање, измену и брисање корисничких налога путем одговарајућег едитора, као и доделу рола и индивидуалних пермисија. Управљање корисничким профилима представља један од најосетљивијих делова система, будући да директно утиче на то којим подацима и функционалностима поједини корисници могу приступити. Неовлашћен приступ овој секцији могао би довести до доделе повишених овлашћења неадекватним корисницима, што би потенцијално угрозило безбедност података и стабилност целокупног система.

Секција за управљање пермисијама омогућава дефинисање и измену рола у систему, при чему свака рола представља логички скуп пермисија који одређује ниво приступа функционалностима апликације. Коришћењем концепта рола поједностављено је администрирање већег броја корисника јер се промене у нивоима приступа могу вршити централизовано, без потребе за индивидуалним подешавањима појединачних налога. Овај приступ је од посебног значаја у већим организацијама, где различите организационе јединице имају строго дефинисане надлежности.

Управљање типовима авиона подељено је на активне и историјске типове. Активни типови авиона представљају типове летелица које се тренутно користе у оперативном саобраћају и за које је омогућена измена података или пензионисање путем одговарајућег едитора. Са друге стране, историјски типови представљају типове летелица које су се раније користиле, али више нису у активној употреби. Чување историјских типова авиона има вишеструки значај, јер омогућава анализу развоја флоте, очување историјских података о ранијим операцијама, као и коришћење ових података у аналитичке и статистичке сврхе, без нарушавања актуелних оперативних подешавања система.

Секција за управљање аеродромима омогућава управљање списком подржаних аеродрома који се користе приликом креирања рута. Подаци о аеродромима уносе се и мењају путем одговарајућег едитора, чиме се обезбеђује конзистентност и поузданост података у целом систему и спречава унос некоректних или непотпуних података који би могли утицати на планирање летова.

Оваква организација административних функционалности омогућава висок ниво контроле приступа, јасну сегментацију одговорности и поуздано управљање осетљивим подацима. Истовремено, омогућена је флексибилност и лако проширивање система у складу са будућим потребама авио-компаније.

6.6. Контрола приступа и видљивост функционалности

Контрола приступа представља један од кључних безбедносних механизама развијеног *AMOS* система и заснива се на *permission-based* моделу ауторизације. Овим моделом дефинише се доступност функционалности у зависности од роле и ефективних пермисија пријављеног корисника, чиме се обезбеђује да сваки корисник има приступ искључиво оним деловима система који су неопходни за обављање његових задатака.

Након успешне аутентификације, систем на основу роле корисника и додељених пермисија динамички одређује које функционалности ће бити доступне. Овај механизам примењује се на више нивоа система, укључујући приказ ставки менија, приступ појединачним страницама и могућност извршавања конкретних операција, као што су измена података или приступ административним функционалностима.

Посебна пажња посвећена је контроли видљивости корисничког интерфејса. Корисници у интерфејсу виде искључиво оне опције и модуле за које имају одговарајућа овлашћења, док су недозвољене функционалности у потпуности скривене. Оваквим приступом смањује се сложеност корисничког интерфејса, унапређује корисничко искуство и истовремено смањује могућност ненамерних корисничких грешака.

У великом броју веб апликација, информације о корисничким пермисијама и улогама чувају се на клијентској страни, најчешће у оквиру механизама као што је *local storage*. Иако овакав приступ може поједноставити имплементацију, он потенцијално представља безбедносни ризик, јер подаци смештени на клијентској страни могу бити изложени манипулацији. У развијеном систему овај проблем је избегнут архитектонским избором клијентске апликације реализоване у *Flutter* окружењу, где не постоји отворени механизам складиштења доступан кориснику за измену критичних података о пермисијама.

Контрола приступа се не ослања искључиво на клијентску страну. Сваки захтев упућен серверској апликацији пролази кроз додатну проверу пермисија, чиме се спречава могућност заобилажења безбедносних механизама манипулацијом клијентске апликације. На тај начин обезбеђен је вишеслојни безбедносни модел, у којем су правила приступа доследно примењена како на нивоу корисничког интерфејса, тако и на нивоу серверске логике.

Permission-based модел омогућава и флексибилно проширивање система. Додавањем нових функционалности могу се дефинисати нове пермисије без нарушавања постојеће структуре рола. Ово је од посебног значаја за системе који се развијају и надограђују током дужег временског периода, као што је *AMOS* систем у оквиру комерцијалне авио-компаније.

Примена оваквог модела контроле приступа посебно је значајна у окружењима са великим бројем корисника и различитим нивоима одговорности, где је неопходно прецизно дефинисати ко има приступ одређеним подацима и функционалностима. На овај начин *AMOS* систем обезбеђује поуздану основу за безбедан и контролисан рад у комплексном оперативном окружењу авио-индустрије.

7. Закључак

У оквиру овог рада представљен је развој интелигентног *AMOS* система за потребе комерцијалне авио-компаније, чији је циљ унапређење управљања флотом, рутама, одржавањима и корисничким приступом, уз примену савремених софтверских и интелигентних технологија. Посебан акценат стављен је на модуларност система, безбедност, као и могућност лаког проширивања и прилагођавања будућим захтевима.

Развијени систем имплементиран је као дистрибуирана, сервисно-оријентисана и вишеслојна апликација, у којој су презентациони слој, серверска логика и база података јасно раздвојени. Оваква архитектура омогућава бољу прегледност система, једноставније одржавање и независан развој појединих компоненти. Додатна вредност постигнута је издвајањем интелигентних компоненти у засебне *provider* сервисе, чиме је смањена сложеност главне серверске апликације и повећана поузданост система.

Коришћењем *Flutter* радног оквира реализована је клијентска апликација оптимизована за таблет уређаје, што омогућава употребу система у различитим оперативним окружењима, као што су аеродроми, хангари и оперативни центри. Овакво решење одговара савременим трендовима рада на даљину и повећава доступност система корисницима ван класичног канцеларијског окружења.

Посебан значај у развијеном систему има примена машинског учења. Коришћењем *ML.NET* радног оквира имплементирани су регресиони модели за предикцију трајања летова и процену датума завршетка одржавања. Ове интелигентне функционалности доприносе доношењу информисанијих оперативних одлука, што је од посебне важности у условима савремене авио-индустрије, коју карактеришу повећан обим саобраћаја, загушење ваздушног простора и изазови у ланцима снабдевања резервним деловима.

Безбедност система представљала је један од кључних аспеката развоја. Имплементиран је механизам аутентификације заснован на *JWT* токенима, као и *permission-based* модел контроле приступа који омогућава фино подешавање овлашћења корисника. Динамичка контрола видљивости функционалности у корисничком интерфејсу, у комбинацији са серверском валидацијом сваког захтева, обезбеђује висок ниво заштите осетљивих података и спречава неовлашћен приступ систему.

Током реализације система појавили су се и одређени имплементациони изазови. Један од кључних изазова односио се на пројектовање флексибилног *permission-based* модела ауторизације, који је морао истовремено да обезбеди висок ниво безбедности и довољну прилагодљивост различитим корисничким улогама. Овај изазов превазиђен је комбиновањем рола са индивидуалним дозвољеним и забрањеним пермисијама, као и централизованом серверском провером сваког захтева.

Додатни изазов представљала је интеграција интелигентних компоненти у оквиру дистрибуираног система, посебно у погледу управљања животним циклусом модела машинског учења и обезбеђивања континуитета рада у случају рестартовања сервиса. Овај проблем решен је имплементацијом механизма резервног учитавања модела из претходно генерисаних фајлова, чиме је обезбеђена стабилност система. Са друге стране, одређени изазови, попут потпуне аутоматизације периодичног обучавања модела у продукционом окружењу, нису реализовани у оквиру овог рада. Један од разлога за то јесте чињеница да механизми попут *Hangfire* система подразумевају континуирано покренут сервер и извршавање позадинских задатака у тачно дефинисаним временским интервалима, што би у развојном окружењу захтевало дуготрајан и стабилан рад система. Поред тога, имплементација оваквог решења подразумевала би додатну инфраструктуру и конфигурацију која превазилази оквире и циљеве овог рада.

Развијени *AMOS* систем представља функционално, скалабилно и безбедно решење које може послужити као основа за даљи развој и примену у реалном оперативном окружењу. Иако систем у потпуности испуњава постављене циљеве, постоји простор за будућа унапређења. Потенцијални правци даљег развоја укључују унапређење модела машинског учења коришћењем већих скупова података, увођење додатних аналитичких и оперативних модула, као и интеграцију са екстерним системима авио-компаније.

Закључно, рад показује да је применом савремених технологија и интелигентних метода могуће развити поуздан и флексибилан информациони систем који значајно унапређује управљање сложеним пословним процесима у области авио-индустрије.

Референце

[1] https://docs.flutter.dev Документација радног оквира *Flutter* [08.01.2026.]

[2] https://learn.microsoft.com/en-us/aspnet/core Документација радног оквира *ASP.NET* [22.01.2026.]

[3] https://learn.microsoft.com/en-us/dotnet/machine-learning Документација радног оквира *ML.NET* [08.01.2026.]

[4] https://learn.microsoft.com/en-us/sql Документација *SQL Server* базе података [06.12.2025.]

[5] https://www.jwt.io/introduction Документација *JWT* механизма аутентификације и ауторизације [30.11.2025.]

[6] https://martinfowler.com/microservices/ *Microservices Guide*, M. Fowler [04.01.2026.]

[7] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice, 4th Edition*, O'Reilly, 2021. [21.11.2025.]

[8] R. Michalski, J. Carbonell, T. Mitchell, *Machine Learning: An Artificial Intelligence Approach (Volume I)*, Morgan Kaufmann, 2014. [02.01.2026.]

[9] https://datatracker.ietf.org/doc/html/rfc7519 *RFC 7519: JSON Web Token (JWT)*, IETF [27.11.2025.]

[10] https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html *OWASP Authentication Cheat Sheet* [26.11.2025.]

[11] https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, 2000. [04.01.2026.]

[12] https://csrc.nist.gov/projects/role-based-access-control *Role-Based Access Control (RBAC)*, NIST Computer Security Resource Center [29.11.2025.]

[13] https://docs.hangfire.io Документација *Hangfire* система за позадинске процесе у *.NET* радном окружењу [25.01.2026.]

[14] https://www.sciencedirect.com/science/article/abs/pii/S0969699722001582 A. Abdelghany, V. Guzhva, K. Abdelghany, *The limitation of machine-learning based models in predicting airline flight block time*, Journal of Air Transport Management – Volume 107, 2023. [02.01.2026.]

[15] https://github.com/LukaKaludjerovic/Aviation_Management_System Изворни код развијеног интелигентног *AMOS* система (клијентска и серверска апликација)